

Java Programming Standards and Naming Conventions

A Beginner's Guide to Writing Professional Java Code

Course: Core Java Programming

1. Why Coding Standards Matter?

Imagine reading a book where:

- Every page uses a different font.
- There are no paragraphs.
- There are no punctuation marks.

It would be difficult to understand.

Programming is the same.

Coding standards help developers:

- Improve readability
- Improve maintainability
- Reduce bugs
- Enable team collaboration
- Write industry-standard code

Fact: Large organizations such as Google, Oracle, Amazon, Microsoft, and IBM follow strict coding standards. Poorly written code is often rejected during code reviews.

2. Java Program Structure

A Java program generally follows this structure:

Package Declaration

Import Statements

Class Declaration

Static Variables

Instance Variables

Constructors

Methods

Main Method

Example

```
package com.gla.java;

import java.util.Scanner;

public class Student {

    private int rollNo;
    private String name;

    public Student() {

    }

    public Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }

    public void displayStudent() {
        System.out.println(rollNo + " " + name);
    }

    public static void main(String[] args) {

    }

}
```

3. Naming Conventions

A. Package Names

Rule

- Always lowercase.
- Reverse domain naming.

Correct

`com.company.project`

`com.gla.java`

`org.spring.demo`

Incorrect

`JavaProject`

`MyPackage`

`StudentPackage`

B. Class Names

Rule

Use **Pascal Case (Upper Camel Case)**.

Every word begins with a capital letter.

Correct

`Student`

`Employee`

`BankAccount`

`HospitalManagement`

`StudentInformationSystem`

Incorrect

`student`

employee_data

student_info

Bank_account

C. Interface Names

Follow the same rule as classes.

Correct

Printable

Runnable

Shape

PaymentService

D. Method Names

Rule

Use **camelCase**.

First letter lowercase.

Every new word begins with a capital letter.

Correct

calculateSalary()

displayResult()

findMaximum()

withdrawAmount()

calculateInterest()

Incorrect

CalculateSalary()

calculate_salary()

```
DISPLAY ()
```

```
method1 ()
```

E. Variable Names

Use **camelCase**.

Correct

```
studentName
```

```
employeeSalary
```

```
bankBalance
```

```
totalMarks
```

```
interestRate
```

Incorrect

```
StudentName
```

```
student_name
```

```
x
```

```
abc
```

```
temp1
```

Use meaningful names.

F. Constant Names

Constants use:

- UPPERCASE
- Words separated by underscores

Correct

```
PI
```

MAX_VALUE

DEFAULT_PORT

MINIMUM_BALANCE

Example

```
public static final double PI = 3.14159;
```

G. Object Names

Objects also follow camelCase.

Correct

```
Student student;
```

```
Employee employee;
```

```
Scanner scanner;
```

```
BankAccount account;
```

Incorrect

```
Student Student;
```

```
Scanner Scanner;
```

```
BankAccount BANK;
```

4. Java Keywords

Keywords are reserved words and cannot be used as identifiers.

Examples:

```
class  
public  
private  
static  
void  
if  
else  
switch  
for  
while
```

```
return
new
extends
implements
try
catch
finally
```

Example

□

```
int class = 10;
```

✓

```
int marks = 10;
```

5. Identifiers in Java

Identifiers are names given to:

- Variables
- Methods
- Classes
- Objects
- Packages

Rules

✓ Can contain

- Letters
- Digits
- `_`
- `$`

✓ Cannot

- Start with a digit
- Contain spaces
- Be a keyword

Examples

Valid

`studentName`

`_rollNo`

`salary1`

`$amount`

Invalid

`1student`

`student name`

`class`

`employee-id`

6. Java is Case Sensitive

These are all different identifiers:

`student`

`Student`

`STUDENT`

`StuDent`

7. Braces Style

Recommended

```
if (age >= 18) {  
    System.out.println("Eligible");  
}
```

Avoid

```
if (age>=18)  
{  
    System.out.println("Eligible");  
}
```

8. Indentation

Use **4 spaces** (or one tab consistently).

Correct

```
if (marks >= 40) {  
    System.out.println("Pass");  
}
```

Incorrect

```
if(marks>=40){  
System.out.println("Pass");  
}
```

9. Comments

Single-line Comment

```
// Calculate total marks
```

Multi-line Comment

```
/*  
This program calculates  
student percentage.  
*/
```

Documentation Comment (JavaDoc)

```
/**  
 * Calculates the area of a circle.  
 */
```

10. Whitespace

Good

```
int sum = a + b;
```

Bad

```
int sum=a+b;
```

11. One Statement per Line

Correct

```
int a = 10;  
int b = 20;
```

Avoid

```
int a=10; int b=20;
```

12. Naming Boolean Variables

Use meaningful names.

Correct

```
isValid  
isActive  
hasPermission  
canVote  
isPrime
```

Avoid

```
flag  
x  
status1
```

13. Method Naming Best Practices

Methods should start with a verb.

Examples

```
calculateSalary()
```

```
displayResult()
```

```
printInvoice()
```

```
withdraw()
```

```
deposit()
```

```
validateUser()
```

```
generateReport()
```

14. Class Naming Best Practices

Classes represent nouns.

Examples

```
Student
```

```
Employee
```

```
Hospital
```

```
BankAccount
```

```
Customer
```

```
Product
```

Avoid

```
Calculate
```

```
Display
```

```
Print
```

15. Variable Naming Best Practices

Avoid

a

b

x

temp

abc

Prefer

studentAge

employeeSalary

accountBalance

courseName

16. Magic Numbers

Avoid

```
salary = salary + 5000;
```

Use constants

```
final int BONUS = 5000;
```

```
salary += BONUS;
```

17. Java File Naming Rule

Class Name

Student

File Name

Student.java

Never

```
student.java
```

```
Student1.java
```

18. Output Formatting

Bad

```
System.out.println(name+age+maks);
```

Good

```
System.out.println("Name : " + name);  
System.out.println("Age : " + age);  
System.out.println("Marks: " + marks);
```

19. Common Beginner Mistakes

- Missing semicolon (;)
 - Incorrect capitalization (`String` vs `string`)
 - Using `=` instead of `==`
 - Unmatched braces { }
 - Forgetting to initialize variables
 - Using poor variable names (`a`, `x`, `temp`)
 - Not following indentation
 - Ignoring compiler warnings
-

20. Java Coding Checklist

Before submitting your program, verify that:

- ✓ Class name starts with an uppercase letter.
- ✓ Method names use camelCase.
- ✓ Variable names are meaningful.
- ✓ Constants use uppercase with underscores.
- ✓ Proper indentation is maintained.
- ✓ Comments explain complex logic.

- ✓ No unnecessary spaces or blank lines.
 - ✓ Braces are properly aligned.
 - ✓ One statement appears on each line.
 - ✓ The program compiles without warnings or errors.
-

Industry Best Practices

- Follow the **DRY (Don't Repeat Yourself)** principle.
 - Keep methods short and focused on a single task.
 - Use descriptive names instead of abbreviations.
 - Write code for readability first, optimization later.
 - Validate user input before processing.
 - Avoid hard-coded values; use constants or configuration.
 - Remove unused variables, imports, and commented-out code before submission.
 - Follow Java's standard coding conventions consistently across all projects.
-

Classroom Activity

Kindly identify and correct the issues in the following code:

```
public class student {  
  
    public static void Main(String Args[]) {  
  
        int A=10;  
        int B=20;  
        int C=A+B;  
        System.out.println(C);  
  
    }  
  
}
```

Expected Improvements

- Rename the class to `Student`.
- Change `Main` to `main`.
- Change `Args` to `args`.
- Use meaningful variable names such as `firstNumber`, `secondNumber`, and `sum`.
- Apply proper indentation and spacing.
- Follow standard Java formatting conventions.

Quick Reference Table

Element	Naming Convention	Example
Package	lowercase	com.gla.java
Class	PascalCase	StudentManagement
Interface	PascalCase	PaymentService
Method	camelCase	calculateSalary()
Variable	camelCase	studentName
Object	camelCase	student
Constant	UPPER_CASE	MAX_STUDENTS
File	Same as Class	Student.java

Practice Exercises

1. Identify and correct the naming convention errors in a given Java program.
2. Rename poorly named variables and methods to meaningful names.
3. Write a Java program that follows all Java coding standards discussed.
4. Review a classmate's code and list at least five improvements related to formatting and naming.
5. Create a `StudentManagement` class using proper package naming, class naming, variable naming, methods, constants, comments, and indentation.

This document provides students with a strong foundation in Java syntax, coding standards, and professional programming practices before they begin writing larger Core Java programs. It also aligns with conventions used in enterprise Java development and code review processes.